

Bazele Inteligenței Artificiale

Lab 7. Manipularea datelor în PyTorch

Efrem Dragoș-Sebastian-Mihaly

Ce este PyTorch?

- Un framework open-source pentru machine learning și deep learning
- Dezvoltat de Facebook AI (acum Meta AI)
- Foarte popular pentru cercetare și dezvoltare rapidă a modelelor de inteligență artificială
- Ușor de utilizat și învățat, mai ales pentru cei care sunt deja familiari cu Python
- Comunitate activă, multe tutoriale și resurse online.

Ce este PyTorch?

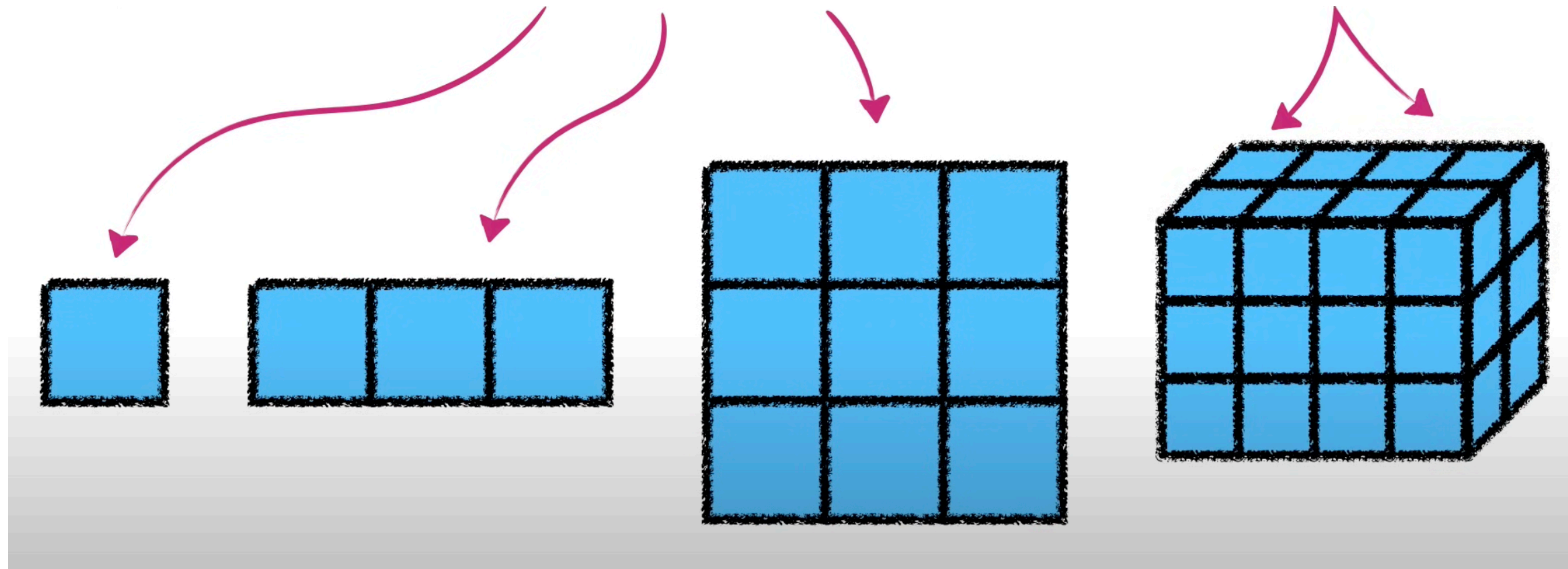
- PyTorch are la bază conceptul de tensor
- Conceptul de tensor există și în fizică și matematică, însă definiția lui diferă față de definiția pe care o folosim în machine learning
- Deci, nu vă îngrijorați dacă găsiți diverse explicații pentru conceptul de tensor, noi ne vom focaliza doar pe cel folosit în machine learning

Conceptul de tensor

- Structuri de date multi-dimensionale care stochează informații numerice
- Generalizare a scalarilor (un singur număr), vectorilor (listă de numere) și matricelor (~~tablouri de numere~~) la dimensiuni mai mari
- Tipuri de tensori:
 - Scalar: Tensor de dimensiune 0 (ex: 5 sau 3.14)
 - Vector: Tensor de dimensiune 1 (ex: [1, 2, 3])
 - Matrice: Tensor de dimensiune 2 (ex: [[1, 2, 3], [4, 5, 6]])
 - Tensor 3D+: Mai multe dimensiuni, ca un “cub” sau colecție de matrici (ex: o imagine RGB de dimensiunea: height × width × channels) unde channels = 3

Conceptul de tensor

- Totuși, tensorii reprezintă mai mult decât o simplă generalizare a scalarilor, vectorilor și matricelor
- În Numpy, există ndarrays, care prezintă și el o modalitate de a crea structuri multi-dimensionale la fel ca tensorii. Atunci, de ce să folosim tensori?



Conceptul de tensor

- Tensorii au un avantaj enorm când vine vorba de:

Accelerare hardware

Accelerare hardware

- Un dezavantaj al NumPy este că funcționează doar pe CPU
- De ce nu ne place să facem operații pe CPU?
- Pe CPU, toate operațiile se efectuează secvențial
- E destul de încet, mai ales când în ML lucrăm cu o tonă de operații pe matrice și matricele sunt de obicei foarte mari
- Chiar dacă are mai multe nuclee, numărul acestora este limitat (de obicei 4-16 nuclee)

Accelerare hardware

- Soluție: Folosim GPU-uri
 - GPU-urile sunt optimizate pentru paralelizare masivă
 - Au mii de nuclee care pot lucra în același timp
 - Fiecare nucleu GPU procesează bucăți mici ale tensorilor, în paralel

Accelerare hardware

- Haideți să vedem un exemplu concret pe adunarea a două matrice:

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}, \quad B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}, \quad C = \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix}$$

Adunare de matrice: $C = A + B$

$$C = \begin{bmatrix} a_{11} + b_{11} & a_{12} + b_{12} \\ a_{21} + b_{21} & a_{22} + b_{22} \end{bmatrix}$$

Accelerare hardware

$$C = \begin{bmatrix} a_{11} + b_{11} & a_{12} + b_{12} \\ a_{21} + b_{21} & a_{22} + b_{22} \end{bmatrix}$$

- Pe CPU:
 - CPU calculează fiecare operație secvențial
 - Prima dată calculează $C[0][0] = a_{11} + b_{11}$
 - Abia după ce acel rezultat a fost obținut, calculează $C[0][1] = a_{12} + b_{12}$
 - CPU-ul își poate folosi nucleele pentru a realiza aceste operații în paralel, dar ele fiind reduse (4-16) puterea lor scade pe măsură ce dimensiunea matricei crește (de exemplu dacă avem o matrice 10,000 x 10,000, operația pe CPU devine lentă până și cu 16 nuclee)

Accelerare hardware

$$C = \begin{bmatrix} a_{11} + b_{11} & a_{12} + b_{12} \\ a_{21} + b_{21} & a_{22} + b_{22} \end{bmatrix}$$

- Pe GPU:
 - GPU-ul împarte matricele în bucăți mici (fiecare element este o “sarcină”)
 - Fiecare nucleu GPU procesează un element în paralel:
 - Nucleul 1 calculează $c_{11}=a_{11}+b_{11}$, nucleul 2 calculează $c_{12}=a_{12}+b_{12}$, nucleul 3 calculează $c_{21}=a_{21}+b_{21}$, nucleul 4 calculează $c_{22}=a_{22}+b_{22}$
 - GPU poate avea mii de nuclee, deci chiar și pentru matrice mari (ex. 10,000 x 10,000), operația este rapidă

Accelerare hardware

- De ce CPU-urile au puține nuclee pe când GPU-urile pot avea mii de nuclee?
- CPU-ul:
 - Este proiectat să fie un „generalist”, să execute o gamă largă de sarcini diverse, inclusiv cele complexe care necesită decizii și ramificații logice
 - Focusul este pe performanța per nucleu și capacitatea de a gestiona câteva fire de execuție în același timp
- GPU-ul:
 - Este un „specialist”, proiectat pentru a procesa volume mari de date în paralel, mai ales operații repetate și simple, cum ar fi cele matematice (de exemplu, înmulțirea matricilor pentru grafica 3D sau machine learning)
 - Focusul este pe paralelism masiv, ceea ce înseamnă că are mii de nuclee mai simple, dar eficiente pentru calcule repetitive

Accelerare hardware

- De ce CPU-urile au puține nuclee pe când GPU-urile pot avea mii de nuclee?

Răspuns:

- GPU-urile au mii de nuclee pentru că fiecare nucleu este mic, simplu și dedicat unei părți a unei probleme masive și a căror operații se pot realiza independent
- CPU-urile au mai puține nuclee pentru că fiecare este mare, puternic și proiectat pentru sarcini complexe și diverse

Accelerare hardware

- Ce alte operații des folosite pe matrice sunt paralelizabile GPU?
 - Înmulțirea
 - Convoluția

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}, \quad B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}, \quad C = \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix}$$

Înmulțire de matrice: $C = A \cdot B$

- $c_{11} = a_{11} \cdot b_{11} + a_{12} \cdot b_{21}$
- $c_{12} = a_{11} \cdot b_{12} + a_{12} \cdot b_{22}$
- $c_{21} = a_{21} \cdot b_{11} + a_{22} \cdot b_{21}$
- $c_{22} = a_{21} \cdot b_{12} + a_{22} \cdot b_{22}$

Accelerare hardware

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}, \quad B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}, \quad C = \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix}$$

Înmulțire de matrice: $C = A \cdot B$

- $c_{11} = a_{11} * b_{11} + a_{12} * b_{21}$
 - $c_{12} = a_{11} * b_{12} + a_{12} * b_{22}$
 - $c_{21} = a_{21} * b_{11} + a_{22} * b_{21}$
 - $c_{22} = a_{21} * b_{12} + a_{22} * b_{22}$
 - Pentru un CPU, fiecare element al matricei rezultate e calculat pas cu pas, mai întâi c_{11} , după c_{12} , după c_{21} și la final c_{22}
 - GPU-ul alocă un nucleu separat pentru fiecare element din c , deci c_{11}, c_{12}, c_{21} și c_{22} vor fi calculați în același timp
- 

Accelerare hardware

- Convoluția e o altă operație utilă care poate fi paralelizabilă pe un GPU
- Vom vorbi despre ea mai în detaliu laboratoarele viitoare, dar vă dau doar un mic teaser
- Convoluția presupune aplicarea unei matrici filtru (numit kernel) K asupra unei matrici de intrare

3_0	3_1	2_2	1	0
0_2	0_2	1_0	3	1
3_0	1_1	2_2	2	3
2	0	0	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

Conceptul de tensor

- Încă un avantaj enorm al tensorilor în PyTorch:

Automatic Differentiation

Automatic Differentiation

- Presupune calculul derivatelor parțiale ale unei funcții și obținerea valorii numerice
- De exemplu, fie următoarea funcție: $f(x, y) = (x + y) \cdot (x - y)$
- Să zicem că vrem să calculăm derivata parțială în funcție de x în punctul $x = 2$
- De asemenea, să zicem că vrem să calculăm derivata parțială în funcție de y în punctul $y = 5$

Automatic Differentiation

- De exemplu, fie următoarea funcție: $f(x, y) = (x + y) \cdot (x - y)$
- Să zicem că vrem să calculăm derivata parțială în funcție de x în punctul $x = 2$
- De asemenea, să zicem că vrem să calculăm derivata parțială în funcție de y în punctul $y = 5$

```
✓ [9] import torch
0 sec.
x = torch.tensor(2.0, requires_grad=True)
y = torch.tensor(5.0, requires_grad=True)
f = (x + y) * (x - y)

✓ [10] f.backward()
0 sec.

✓ [11] x.grad, y.grad
0 sec.
⇒ (tensor(4.), tensor(-10.))
```

Automatic Differentiation

```
✓ [9] import torch
0
sec.
x = torch.tensor(2.0, requires_grad=True)
y = torch.tensor(5.0, requires_grad=True)

f = (x + y) * (x - y)

✓ [10] f.backward()
0
sec.

✓ [11] x.grad, y.grad
0
sec.
⇒ (tensor(4.), tensor(-10.))
```

$$\frac{\partial f}{\partial x} = \frac{\partial}{\partial x}((x + y)(x - y)) = 2x$$

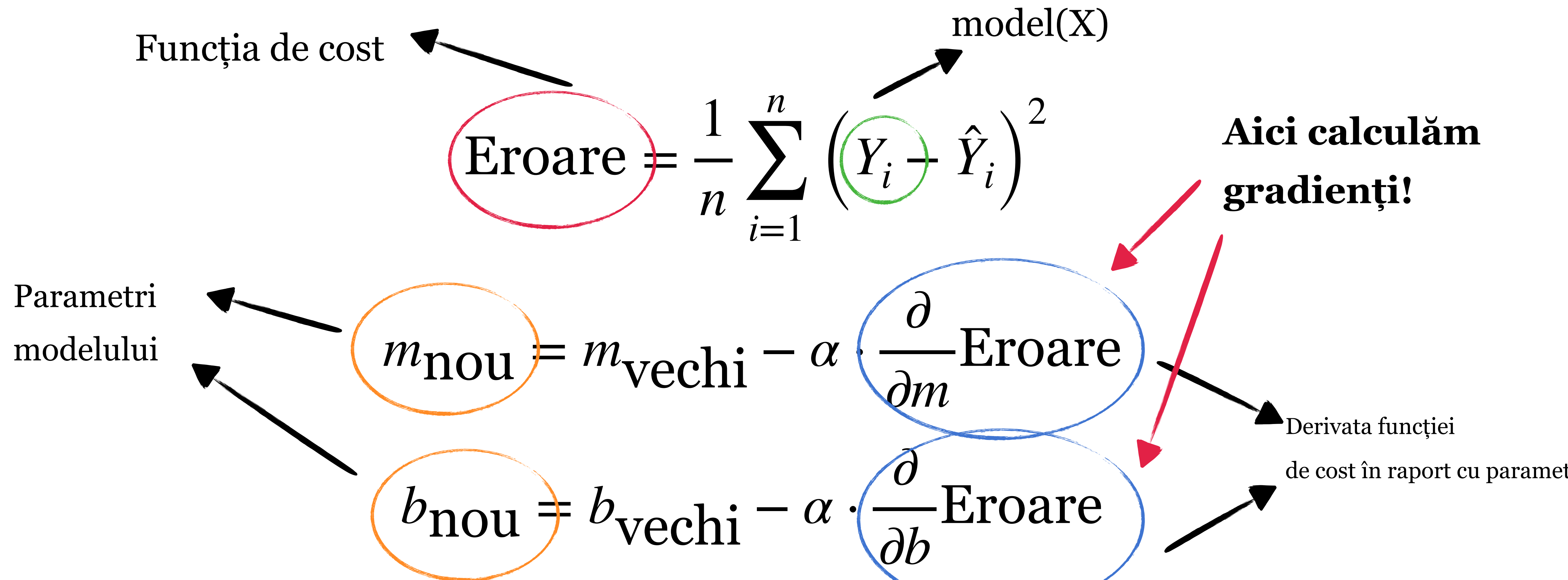
$$\left. \frac{\partial f}{\partial x} \right|_{x=2} = 2(2) = 4$$

$$\frac{\partial f}{\partial y} = \frac{\partial}{\partial y}((x + y)(x - y)) = -2y$$

$$\left. \frac{\partial f}{\partial y} \right|_{y=5} = -2(5) = -10$$

Automatic Differentiation

- Totuși, în ce caz e util să obținem derivata parțială a unei funcții în raport cu niște variabile?
- Haideți să revedem gradient descent



Automatic Differentiation

- În Gradient Descent, avem nevoie de derivata funcției de cost în raport cu fiecare dintre parametri
- Folosim aceste derivate pentru a actualiza parametri modelului astfel încât să minimizăm funcția de cost
- Dacă f era funcția noastră de cost, iar x și y erau parametri modelului nostru, astfel am fi obținut gradientii necesari pentru minimizarea funcției de cost
- Funcția `backward()` reprezintă, de fapt, o implementare a backpropagation dar care se ocupă de mai multe lucruri behind the scenes

```
✓ [9] import torch
0 sec.
x = torch.tensor(2.0, requires_grad=True)
y = torch.tensor(5.0, requires_grad=True)

f = (x + y) * (x - y)

✓ [10] f.backward()
0 sec.

✓ [11] x.grad, y.grad
0 sec.
⇒ (tensor(4.), tensor(-10.))
```


Automatic Differentiation

```
[9] import torch

x = torch.tensor(2.0, requires_grad=True)
y = torch.tensor(5.0, requires_grad=True)

f = (x + y) * (x - y)

[10] f.backward()

[11] x.grad, y.grad
(tensor(4.), tensor(-10.))

f = (x + y) * (x - y)
f.backward()
x.grad, y.grad
(tensor(8.), tensor(-20.))
```

- În PyTorch, gradientii se acumulează la fiecare apel de backward()
- Dacă vrem ca acest lucru să nu se întâmple, trebuie să îi resetăm

Automatic Differentiation

```
[9] import torch

x = torch.tensor(2.0, requires_grad=True)
y = torch.tensor(5.0, requires_grad=True)

f = (x + y) * (x - y)

[10] f.backward()

[11] x.grad, y.grad
(tensor(4.), tensor(-10.))

f = (x + y) * (x - y)
f.backward()
x.grad, y.grad
(tensor(8.), tensor(-20.))

[15] x.grad.zero_()
y.grad.zero_()

f = (x + y) * (x - y)

f.backward()

x.grad, y.grad
(tensor(4.), tensor(-10.))
```

- Resetarea unui gradient se face cu `.grad.zero_()`

Automatic Differentiation

- De ce ar fi util acumularea gradientilor?
- Să ne reamintim puțin mini-batch gradient descent
- Acolo, actualizarea parametrilor se realiza iterativ, la fiecare batch, în funcție de un subset din date
- Dacă batch-ul este prea mic, poate exista zgomot în actualizarea parametrilor (cine știe, avem un outlier sau ~~datele sunt prea complexe~~)
- Dacă batch-ul este mare, actualizarea parametrilor este mai precisă, pentru că modelul învață din mai multe date în același timp și ~~zgomotele se pot reduce~~
- Există o metodă prin care putem face uz de precizia obținută având batch-uri mai mari chiar dacă avem batch-uri mici (din cauza limitărilor de memorie)

Automatic Differentiation

- Există o metodă prin care putem face uz de precizia obținută având batch-uri mai mari chiar dacă avem batch-uri mici (din cauza limitărilor de memorie)
- Metoda presupune acumularea gradientilor pe parcursul mai multor batch-uri mici, practic însumarea acestora, și actualizarea parametrilor ~~abia după ce s-au procesat un număr suficient de mare de batch-uri~~
- Practic, astfel încercăm să simulăm efectul obținut din batch-uri mai mari, având batch-uri mai mici
- Lucrul acesta duce la o antrenare mai stabilă și permitere obținerea unei performanțe foarte bune atunci când resursele de memorie sunt limitate

That's it for today!
Now let's do some exercises.

That's it for today!

Now let's do some exercises.

Laboratory 7 – Exercises

1. Create tensors of the following ranks: 1, 2, and 3. Use values between 0 and 1 to fill your tensors. The size of the tensors can be defined as you wish, given that the ranks are created correctly. Print the shape of each of the tensors using the `shape` property.

2. The **sigmoid** activation function is an essential function in neural networks and is defined as follows:

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}.$$

Take the derivative of this function with respect to x , where $x = [-10, 10, 8]$.

3. Define the following two matrices:

$$A = \begin{bmatrix} 1 & 2 & -3 \\ 4 & 5 & 10 \end{bmatrix}, \quad B = \begin{bmatrix} 10 & 11 & -2 \\ 13 & -3 & 8 \end{bmatrix}.$$

- calculate the Hadamard product of the two matrices;
- add the scalar $a = 10$ to each element of matrix B ;
- print the transpose of both matrices.

4. Create the following two matrices:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \quad B = \begin{bmatrix} 10 & 11 \\ -2 & 0 \\ 1 & -3 \end{bmatrix}.$$

Multiply the two matrices and compute the Frobenius norm of A and B .